Revision — 1.2.1 Operating Systems

OCR H446 — one-page revision summary

Must know

- An **operating system (OS)** is software that manages the hardware and provides a platform for applications, using **abstraction** to hide complexity. Core roles: **memory management, processor scheduling, interrupt handling, I/O & peripheral management (drivers), user interface, security & file management**.
- **Memory management — paging vs segmentation:** paging uses **fixed-size** blocks (pages ↔ frames, divided by the OS) and suffers **internal** fragmentation; segmentation uses **variable-size** logical blocks (divided by the program's logical structure) and suffers **external** fragmentation.
- **Virtual memory** uses **secondary storage** (a swap/page file) *as if it were RAM* so more/larger programs can run — it is **slower** than real RAM and does not add physical memory. A **page fault** occurs when a needed page is on disk; excessive swapping causes **disk thrashing**, slowing the machine to a crawl.
- **Interrupt** = a signal (hardware or software) that a device/process needs the processor's attention; it removes the need for **polling** and interrupts have **priorities**. The **ISR (interrupt service routine)** is the OS code that handles a specific interrupt.
- **Interrupt handling (in order):** CPU checks the interrupt register at the **end of each F–D–E cycle** → finishes the current instruction (if the interrupt is higher priority) → **pushes registers (PC, ACC) onto the stack** → loads and runs the correct **ISR** → **pops registers off the stack** to resume. The **stack (LIFO)** allows nested interrupts to resume in the right order.
- **Scheduling** creates the illusion of simultaneous processes and aims for maximum CPU use, fairness and good response. Know: **Round robin** (fixed time slice/quantum, pre-emptive, fair), **FCFS** (arrival order, non-pre-emptive, convoy effect), **Shortest Job First**, **Shortest Remaining Time** (pre-emptive), **Multi-level feedback queue**.
- **OS types: Distributed** (one system across networked computers), **Embedded** (single device, limited resources, held in ROM), **Multitasking** (time-slicing), **Multi-user** (shared resources via scheduler + permissions), **Real-time / RTOS** (guarantees a response within a set deadline, e.g. pacemaker, ABS).
- **BIOS** is firmware on ROM: the **first program to run** at start-up — runs **POST** (Power-On Self-Test), initialises hardware, then loads (bootstraps) the OS into RAM. The BIOS *loads* the OS; it is **not** the OS.
- **Device driver** = software that lets the OS control a specific hardware device by translating OS commands into device instructions; it is **hardware- and OS-specific**.
- **Virtual machine** = software emulating a whole computer. **System VM** runs a whole guest OS (testing/sandboxing/legacy); **process VM** runs platform-independent **intermediate code** (e.g. Java bytecode on the JVM) — portable ("write once, run anywhere") but slower than native.

Key terms

Term	Definition
Paging	Dividing memory into **fixed-size** blocks (pages/frames); causes internal fragmentation
Segmentation	Dividing memory into **variable-size** logical blocks (segments); causes external fragmentation
Virtual memory	Using secondary storage as if it were RAM to run more/larger programs
Disk thrashing	Excessive paging between RAM and disk that slows the system down
Interrupt	A signal that a device/process needs the processor's attention
ISR	OS code run to deal with a particular interrupt
Round robin	Scheduling giving each process a fixed time slice (quantum) in turn
Real-time OS	An OS that guarantees a response within a set time limit

Worked example / how to — handling an interrupt

1. At the **end of the current F–D–E cycle** the CPU checks the interrupt register.
2. If the interrupt is higher priority than the current task, finish the current instruction.
3. **Push** the current register contents (PC, ACC) onto the **stack**.
4. Load the address of the correct **ISR** and execute it.
5. When finished, **pop** the saved registers off the stack and resume the interrupted task (LIFO order supports nested interrupts).

Exam tips

- Paging vs segmentation: the discriminator is **fixed-size (paging)** vs **variable-size/logical (segmentation)** — state it explicitly, plus the fragmentation type.
- Virtual memory does **not** add RAM — say it uses **secondary storage** *as if* it were RAM (slower) and mention **thrashing**.
- For interrupts, always mention that registers are saved on a **stack** so the task can resume — omitting the stack loses marks.
- Match the **OS type to the scenario** and justify (e.g. washing machine → embedded; mainframe → multi-user; ABS → real-time).